

frontend system design

Below is a complete frontend design specification for NCPS, written so an AI agent or engineer can implement the UI, state flow, and API integration deeply and correctly.

NCPS Frontend Architecture Design

1. Frontend Philosophy

The frontend is not just a UI—it is a real-time visualization of system trust and credibility.

It must:

- reflect dynamic credibility changes
- show locality-aware content
- avoid misleading users with raw engagement metrics
- expose system signals (credibility, urgency, proximity) in interpretable form

2. High-Level Architecture

[UI Components] ↓ [State Management Layer] ↓ [API Service Layer] ↓ [Backend APIs]

3. Technology Stack (Recommended)

Framework → React (Next.js preferred) State Mgmt → Zustand / Redux Toolkit UI → TailwindCSS / Material UI
Maps → Mapbox / Google Maps Realtime → WebSockets / SSE

4. Core Frontend Modules

4.1 Feed Module

Purpose

Displays posts filtered by:

- proximity
- credibility

- propagation radius

Data Model

type Post = { post_id: string content: string credibility: number // $C_j(t)$ urgency: number // $U_j(t)$ variance: number // $Var_j(t)$ distance: number proximity: number // $Prox(u,j,t)$ radius: number timestamp: string }

Rendering Logic

show_post if: distance $\leq$ radius AND credibility $\geq$ threshold

UI Representation

- credibility bar (0 $\rightarrow$ 1)
- urgency indicator (color-coded)
- stability indicator (variance)

4.2 Post Component

Structure

[Content] [Credibility Score] [Urgency Tag] [Vote Buttons] [Location Tag]

Behavior

- voting updates backend
- UI updates optimistically
- credibility refreshes asynchronously

Example

"Fire near downtown area" Credibility: 0.82 Urgency: High Distance: 1.2 km

4.3 Voting Component

UI

[👍] [👎]

Logic

onVote(v): disable buttons send API request update local state re-enable after response

Anti-spam UX

- prevent rapid clicking
 - show cooldown
-

4.4 Map Module

Purpose

Visualize spatial propagation.

Features

- user location marker
 - post origin marker
 - propagation radius (circle)
-

Data

type MapPost = { lat: number lon: number radius: number credibility: number }

Rendering

draw circle: center = post location radius = $r_j(t)$ opacity = credibility

4.5 Alert System (Frontend)

Behavior

- receives alerts via WebSocket
 - shows popup notification
-

UI

⚠ Alert "Possible fire reported nearby" Credibility: 0.88 Distance: 0.9 km

Logic

onAlert(event): if not muted: show notification

4.6 User Dashboard

Displays

Reliability Score (R_i) Experience Score (Exp_i) Trust Score (T_i) Anomaly Score ($Anom_i$) Location Confidence (L_i)

Purpose

- transparency
 - user awareness
 - trust-building
-

4.7 Location Module

Behavior

get user location periodically send to backend update map + feed

Privacy Control

- toggle location sharing
 - show confidence level
-

5. State Management Design

5.1 Global State

```
type GlobalState = { user: UserState posts: Post[] alerts: Alert[] location: Location }
```

5.2 User State

```
type UserState = { id: string trust: number reliability: number exp: number anomaly: number  
location_confidence: number }
```

5.3 State Flow

API Response → Store Update → UI Re-render

6. API Service Layer

6.1 API Functions

```
createPost(content) votePost(post_id, vote) getFeed(location) updateLocation(lat, lon) subscribeAlerts()
```

6.2 Example

```
async function votePost(postId, vote) { return fetch('/post/vote', { method: 'POST', body: JSON.stringify({  
postId, vote }) }) }
```

7. Real-Time Updates

7.1 WebSocket Channels

/alerts /post-updates /user-updates

7.2 Event Types

```
{ "type": "POST_UPDATE", "post_id": "...", "credibility": 0.85 }
```

7.3 Update Flow

onMessage: update store re-render UI

8. Feed Ranking Logic (Frontend)

Score Function

$\text{score} = w_1 * \text{credibility} + w_2 * \text{proximity} + w_3 * \text{urgency}$

Sorting

sort posts by score descending

9. UI Design Principles

9.1 Avoid Raw Engagement Metrics

Do NOT show:

- likes count
- share count

Instead show:

- credibility
 - trust indicators
-

9.2 Color Encoding

Credibility: Green → High Yellow → Medium Red → Low Urgency: Red → Critical Orange → Medium

9.3 Transparency

Always expose:

- why a post is shown
 - credibility explanation
-

10. Error Handling

Cases

- API failure

- location denied
- vote rejected

Strategy

try: call API catch: show fallback message

11. Performance Optimization

Techniques

- virtualized lists (large feeds)
- memoized components
- debounce location updates

12. Security Considerations

Prevent

- spam voting (frontend throttle)
- fake location injection (basic checks)

Trust backend for:

- final validation
- scoring

13. Minimal Page Structure

Pages

/ → Feed /post/:id → Post details /map → Spatial view /dashboard → User stats

14. End-to-End Flow

User opens app ↓ Location fetched ↓ Feed requested ↓ Posts displayed ↓ User votes ↓ API call ↓ State updates ↓ UI refresh ↓ Alerts received (if any)

15. Final Insight

The frontend must reflect:

credibility-driven information, not engagement-driven information

This is a fundamental shift from traditional systems.
